

The Complete Stack Roadmap (Beginner → Advanced, Java)

72 curated problems, prioritizing **LeetCode (LC)**, supplemented with **GeeksforGeeks (GfG)** for classic patterns LeetCode doesn't cover well (Infix/Postfix/Prefix conversions, Stock Span, Celebrity Problem, Previous/Next Smaller Element as standalone drills). As with the Linked List roadmap: **only LeetCode assigns numeric problem IDs** — GfG problems are identified by title only, shown as "—" in the Problem # column. Every title below was checked against GfG's actual practice-page listings at the time of writing.

Tags: 🔥 Must-Do · ☆ Recommended · ⚙️ Optional · 🏆 Hard Challenge · 📌 Premium

Level 0 — Stack Fundamentals & Basic Operations

Patterns: Stack Fundamentals, Basic Stack Operations

Before any problems: **implement a stack yourself twice** — once backed by an array (with resizing), once backed by a singly linked list (push/pop at the head). This isn't a platform problem, it's the actual prerequisite; skipping it is the #1 reason people fumble "design a stack with extra property X" questions later.

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Pre
1	LeetCode	20	Valid Parentheses	Easy	Basic Stack Ops	Build own

2	LeetCode	225	Implement Stack using Queues	Easy	Basic Stack Ops (mechanics)	#1
3	LeetCode	232	Implement Queue using Stacks	Easy	Basic Stack Ops (mechanics)	#2
4	GfG	—	Stack using Queues	Easy	rep of #2	#2
5	GfG	—	Queue using Stacks	Easy	rep of #3	#3

Level 1 — Basic Easy Problems (Stack + Arrays)

Pattern: Stack + Arrays

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prereq
6	LeetCode	682	Baseball Game	Easy	Stack + Arrays (simulate ops)	Level 0
7	LeetCode	844	Backspace String Compare	Easy	Stack + Arrays (contrast: also solvable w/ Two Pointers)	Level 0

8	LeetCode	946	Validate Stack Sequences	Medium	Stack Simulation	Level 0
---	----------	-----	--------------------------	--------	------------------	---------

Level 2 — Stack + Strings

Pattern: Stack + Strings, Decode String / Nested Structures

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	P
9	LeetCode	1047	Remove All Adjacent Duplicates In String	Easy	Stack + Strings	L
10	LeetCode	1544	Make The String Great	Easy	Stack + Strings (rep of #9's idea)	#
11	LeetCode	1209	Remove All Adjacent Duplicates in String II	Medium	Stack + Strings (count-aware, store {char,count} pairs)	#
12	LeetCode	71	Simplify Path	Medium	Stack + Strings (path token parsing)	#
13	LeetCode	394	Decode String	Medium	Nested Structures / Recursion+Stack	L
14	LeetCode	388	Longest Absolute File Path	Medium	Stack Simulation (depth tracking)	#

15	LeetCode	385	Mini Parser	Medium	Nested Structures	#
16	LeetCode	341	Flatten Nested List Iterator	Medium	Nested Structures + Design	#

Level 3 — Parentheses / Expression Problems

Patterns: Valid Parentheses/Balanced Brackets, Expression Evaluation, Infix/Prefix/Postfix

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	F
17	GfG	—	Check for balanced parentheses	Easy	Balanced Brackets (rep of LC 20)	L
18	LeetCode	921	Minimum Add to Make Parentheses Valid	Medium	Balanced Brackets (greedy counting)	#
19	LeetCode	1249	Minimum Remove to Make Valid Parentheses	Medium	Balanced Brackets + Reconstruction	#
20	LeetCode	1541	Minimum Insertions to Balance a Parentheses String	Medium	Balanced Brackets (weighted counting)	#

21	LeetCode	1963	Minimum Number of Swaps to Make the String Balanced	Medium	Balanced Brackets (counting)	#
22	LeetCode	32	Longest Valid Parentheses	Hard	Balanced Brackets (index-based stack)	#
23	LeetCode	856	Score of Parentheses	Medium	Nested Structures (recursive scoring)	#
24	LeetCode	1190	Reverse Substrings Between Each Pair of Parentheses	Medium	Nested Structures (stack-of-strings)	#
25	LeetCode	150	Evaluate Reverse Polish Notation	Medium	Postfix Evaluation	L
26	GfG	—	Evaluation of Postfix Expression	Easy	rep of #25	#
27	GfG	—	Infix to Postfix	Medium	Infix/Postfix Conversion	#
28	GfG	—	Postfix to Infix	Easy	Conversion (reverse direction)	#
29	GfG	—	Prefix to Infix	Easy	Conversion	#
30	GfG	—	Infix To Prefix	Medium	Conversion	#

31	LeetCode	227	Basic Calculator II	Medium	Expression Evaluation (+, -, *, / no parens)	#
32	LeetCode	224	Basic Calculator	Hard	Expression Evaluation (+, -, nested parens)	#
33 🐼	LeetCode	772	Basic Calculator III	Hard	Expression Evaluation (full: +, -, *, /, parens)	#

Level 4 — Monotonic Stack (Foundations)

Patterns: Monotonic Increasing Stack, Monotonic Decreasing Stack, Next Greater Element, Next Smaller Element

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	F
34	LeetCode	739	Daily Temperatures	Medium	Monotonic Decreasing Stack 🔥 ARCHETYPE	L
35	GfG	—	Next Greater Element	Medium	Monotonic Stack (classic single-array version)	#
36	LeetCode	496	Next Greater Element I	Easy	Monotonic Stack + HashMap (query version)	#

37	LeetCode	503	Next Greater Element II	Medium	Monotonic Stack (circular array, 2-pass trick)	#
38	GfG	—	Next Smaller Element	Medium	Monotonic Increasing Stack	#
39	GfG	—	Previous Smaller Element	Medium	Monotonic Stack (scan direction flipped)	#
40	LeetCode	2454	Next Greater Element IV	Hard	Monotonic Stack (2nd-next, double stack)	#

Must-Do & key pattern: #34 is the archetype every monotonic-stack problem below traces back to — push indices, and whenever the current value "breaks" the stack's monotonic order, pop and resolve. #35–#39 drill the same core loop in every direction (next/previous, greater/smaller) until you can write it without thinking. Note the mirror-image relationship: "previous greater/smaller" is just "next greater/smaller" run with the scan direction reversed — practice deriving one from the other instead of memorizing four separate templates.

Level 5 — Next Greater/Smaller Applied (Stock Span, Contribution Problems)

Patterns: Stock Span, Previous Greater/Smaller, Stack + Greedy

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	
-------	----------	-----------	--------------	------------	---------	--

41	GfG	—	Stock Span Problem	Medium	Monotonic Stack applied (previous greater element)	
42	LeetCode	901	Online Stock Span	Medium	Monotonic Stack + Design	
43	LeetCode	907	Sum of Subarray Minimums	Medium	Monotonic Stack (contribution technique)	
44	LeetCode	2104	Sum of Subarray Ranges	Medium	Monotonic Stack (max & min contribution)	
45	LeetCode	1856	Maximum Subarray Min-Product	Medium	Monotonic Stack + Prefix Sum	
46	LeetCode	962	Maximum Width Ramp	Medium	Monotonic Stack (build then scan-back)	
47	LeetCode	1130	Minimum Cost Tree From Leaf Values	Medium	Monotonic Stack + Greedy	
48	LeetCode	316	Remove Duplicate Letters	Medium	Monotonic Stack + Greedy (lexicographic)	

49	LeetCode	402	Remove K Digits	Medium	Monotonic Stack + Greedy (build smallest number)
50	LeetCode	1673	Find the Most Competitive Subsequence	Medium	Monotonic Stack + Greedy (k-budgeted)

Must-Do & key pattern: #41/#42 show the "previous greater element" idea solving a real (non-toy) problem — $\text{span} = \text{current index} - \text{index of the previous greater element}$. #43 introduces the **contribution technique**: instead of asking "what's the min of each subarray," ask "for how many subarrays is *this* element the minimum," using its distance to the nearest smaller elements on each side. #48/#49 are the greedy-monotonic-stack archetype: pop while it's safe and beneficial to do so (there's a smaller/better candidate and enough elements remain), otherwise push.

Level 6 — Medium Interview Problems (Design, Recursion+Stack, Stack+HashMap, Stack+LinkedList)

Patterns: Min Stack/Max Stack, Design Stack Problems, Recursion + Stack, Stack + HashMap, Stack + Linked List

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prereq
51	LeetCode	155	Min Stack	Medium	Min Stack Design	Level

52	GfG	—	A Stack with getMin() in O(1) Time	Easy	rep of #51	#51
53 	LeetCode	716	Max Stack	Hard	Max Stack Design (needs O(log n) or amortized popMax)	#51
54	LeetCode	1381	Design a Stack With Increment Operation	Medium	Design (lazy propagation on a stack)	#51
55	GfG	—	Sort a stack using recursion	Medium	Recursion + Stack	Level recurs
56	GfG	—	Reverse a stack using recursion	Medium	Recursion + Stack	#55
57	GfG	—	The Celebrity Problem	Medium	Stack + Greedy Elimination	Level
58	GfG	—	Next Greater Frequency Element	Medium	Stack + HashMap (frequency-aware monotonic stack)	Level HashM
59	LeetCode	636	Exclusive Time of Functions	Medium	Stack + Simulation (call-stack modeling)	Level

60	LeetCode	2487	Remove Nodes From Linked List	Medium	Stack + Linked List (monotonic stack applied to a list)	Level 7
61	LeetCode	173	Binary Search Tree Iterator	Medium	Stack + Recursion (controlled inorder traversal)	Level 7

Must-Do & key pattern: #51 (Min Stack) — keep a second stack (or store pairs) tracking the running minimum alongside each push, so `getMin()` is $O(1)$ without rescanning. #55/#56 force you to solve stack problems *without* a second explicit stack — using the call stack itself as your only extra storage, which is exactly the skill tested by "no extra data structure" interview constraints. #57 (Celebrity Problem) is a genuinely different flavor: use a stack to eliminate candidates pairwise in $O(n)$, a nice bridge between Stack and Greedy reasoning. #60 is the clearest possible example of "Stack + Linked List" — walk the list, but decide what survives using a monotonic stack exactly like #34–#39.

Level 7 — Histogram / Span / Complex Problems

Patterns: Largest Rectangle in Histogram, Trapping Rain Water

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Pre
62	LeetCode	84	Largest Rectangle in Histogram	Hard	Monotonic Stack 🔥 CORE CAPSTONE	Level 7

63	GfG	—	Largest Rectangular Area in a Histogram	Hard	rep of #62	#62
64	LeetCode	85	Maximal Rectangle	Hard	Monotonic Stack (2D: run #62 per row)	#62
65	GfG	—	Maximum Rectangle with all 1s	Hard	rep of #64	#64
66	LeetCode	42	Trapping Rain Water	Hard	Stack (or Two Pointers — solve both ways) 🔥 CORE	Leve
67	LeetCode	407	Trapping Rain Water II	Hard	Heap + BFS (2D extension — different tool, good contrast)	#66

Must-Do & key pattern: #62 is arguably the single hardest "core" (non-exotic) stack problem in interviews: maintain a stack of increasing bar heights; when a shorter bar arrives, pop and compute the rectangle using the popped bar as the height and the current index minus the new stack top minus one as the width. #64 is #62 applied once per row of a binary matrix, treating column-heights as a histogram. #66 (Trapping Rain Water) is worth solving **both** with a stack (track bars, compute trapped water when a taller bar closes a "container") and with the two-pointer/running-max method from the Two Pointers roadmap — comparing them is one of the best ways to internalize when each tool is more natural.

Level 8 — Advanced / Hard Problems

Pattern: Advanced/Hard Stack Problems

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prereqs
68	LeetCode	456	132 Pattern	Medium	Monotonic Stack (non-adjacent triple pattern)	Level 4
69	LeetCode	1944	Number of Visible People in a Queue	Hard	Monotonic Stack	Level 4 Level 5
70	LeetCode	2281	Sum of Total Strength of Wizards	Hard	Monotonic Stack + Prefix Sum (contribution, two layers deep)	Level 5 Prefix
—	LeetCode	772	(cross-ref, Level 3)	Hard	Expression Evaluation capstone	—
—	LeetCode	2454	(cross-ref, Level 4)	Hard	Monotonic Stack capstone	—
—	LeetCode	32	(cross-ref, Level 3)	Hard	Balanced Brackets capstone	—
—	LeetCode	407	(cross-ref, Level 7)	Hard	Heap/BFS capstone	—

Must-Do & key pattern: #68 (132 Pattern) is deceptively hard because the three numbers don't need to be adjacent — the trick is scanning right-to-left, tracking the best "third element candidate" using a monotonic stack, while maintaining a running "second element" threshold. #70 stacks the Level 5 contribution technique on top of prefix sums for a genuinely advanced combination — treat it as the "final exam" of this roadmap.

Stack Pattern Cheat Sheet

When to use a Stack

- Recognize: "most recent," "matching pairs," "nested structure," "undo/backtrack," "process in reverse order of arrival" (LIFO).
- Core idea: last element in is the first one you need back out.

When to use Monotonic Increasing Stack

- Recognize: you need the **previous/next smaller** element for each position, or you're processing bars/heights where a taller-than-expected element should be resolved before continuing.
- Core idea: keep the stack's values in increasing order (bottom to top); pop whenever the incoming element is smaller than the top.

```
Deque<Integer> stack = new ArrayDeque<>(); // stores indices
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && arr[stack.peek()] > arr[i]) {
        int idx = stack.pop();
        // arr[i] is the "next smaller element" for index idx
    }
    stack.push(i);
}
```

When to use Monotonic Decreasing Stack

- Recognize: you need the **previous/next greater** element for each position (Daily Temperatures, Next Greater Element).

- Core idea: keep the stack's values in decreasing order; pop whenever the incoming element is greater than the top.

```
Deque<Integer> stack = new ArrayDeque<>(); // stores indices
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && arr[stack.peek()] < arr[i]) {
        int idx = stack.pop();
        // arr[i] is the "next greater element" for index idx
    }
    stack.push(i);
}
```

Next Greater Element pattern

- Scan left to right; a **decreasing** stack; resolve on a *larger* incoming value.

Next Smaller Element pattern

- Scan left to right; an **increasing** stack; resolve on a *smaller* incoming value.

Previous Greater/Smaller pattern

- Same two templates as above, but scan **right to left** instead of left to right — or equivalently, scan left to right and simply don't pop until you've recorded the top as the "previous" answer for the *current* element before pushing it. Most people find it easier to just reverse the array and reuse the Next-Greater/Smaller template.

Stack vs Queue

- Stack = LIFO (most recent thing matters first) — matching brackets, undo history, DFS.
- Queue = FIFO (oldest thing matters first) — level-order traversal, BFS, task scheduling in arrival order.
- If the problem cares about "the most recent unmatched X," it's a stack. If it cares about "the first thing that arrived," it's a queue.

Stack vs Two Pointers

- Both can solve problems like Trapping Rain Water. Prefer **Two Pointers** when you can maintain a simple running aggregate (like a running max) from each side with $O(1)$ extra space. Prefer a **Stack** when the resolution of an element depends on a variable, unpredictable number of *previous* unresolved elements (e.g., Largest Rectangle in Histogram — you don't know in advance how many previous bars a new short bar will "close out").

Stack vs Sliding Window

- Sliding Window maintains a **contiguous range** with two pointers, expanding/shrinking based on a running condition. A Stack maintains **non-contiguous, LIFO-ordered "unresolved" elements** — the elements on the stack aren't necessarily adjacent in the original array. If the answer only ever depends on a shrinking/growing *contiguous* range, it's Sliding Window; if it depends on "the nearest unresolved bigger/smaller element so far," it's a Stack.

Stack vs Heap

- A Stack gives you $O(1)$ access to the *most recently added* element (LIFO) — no ordering by value. A Heap gives you $O(\log n)$ access to the *min or max by value*, regardless of insertion order. If the problem needs "the largest/smallest so far" independent of when it arrived, use a Heap (see LC 407, Trapping Rain Water II, for a case where a 1D stack solution doesn't generalize and a heap is needed instead).

When to use recursion instead of an explicit stack

- Recursion *is* an implicit stack (the call stack) — use it when the natural definition of the problem is self-referential ("solve the rest, then combine with the current step") and you don't need fine-grained control over the stack's contents mid-traversal.
- Prefer an **explicit** stack when: you need to pause/resume traversal across separate method calls (e.g., LC 173 BST Iterator), you're at risk of a `StackOverflowError` on deep/unbalanced input, or the problem explicitly restricts recursion.

```
// Recursion: reverse a stack without any extra explicit
void reverseStack(Deque<Integer> stack) {
    if (stack.isEmpty()) return;
    int top = stack.pop();
    reverseStack(stack);
    insertAtBottom(stack, top);
}

void insertAtBottom(Deque<Integer> stack, int val) {
    if (stack.isEmpty()) { stack.push(val); return; }
    int top = stack.pop();
    insertAtBottom(stack, val);
    stack.push(top);
}
```

Java Templates for Major Patterns

Balanced Parentheses

```
public boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    Map<Character, Character> pairs = Map.of(')', '(', ')', '[');
    for (char c : s.toCharArray()) {
        if (!pairs.containsKey(c)) {
            stack.push(c);
        } else {
            if (stack.isEmpty() || stack.pop() != pairs.get(c))
                return false;
        }
    }
    return stack.isEmpty();
}
```

Postfix Evaluation

```
public int evalRPN(String[] tokens) {
    Deque<Integer> stack = new ArrayDeque<>();
```

```

for (String tok : tokens) {
    if ("+-*/".contains(tok) && tok.length() == 1) {
        int b = stack.pop(), a = stack.pop();
        stack.push(switch (tok) {
            case "+" -> a + b; case "-" -> a - b;
            case "*" -> a * b; default -> a / b;
        });
    } else {
        stack.push(Integer.parseInt(tok));
    }
}
return stack.pop();
}

```

Monotonic Stack (Next Greater Element)

```

public int[] nextGreaterElement(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    Arrays.fill(res, -1);
    Deque<Integer> stack = new ArrayDeque<>(); // indices
    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
            res[stack.pop()] = nums[i];
        }
        stack.push(i);
    }
    return res;
}

```

Largest Rectangle in Histogram

```

public int largestRectangleArea(int[] heights) {
    Deque<Integer> stack = new ArrayDeque<>();
    int maxArea = 0;
    for (int i = 0; i <= heights.length; i++) {
        int h = (i == heights.length) ? 0 : heights[i];
        while (!stack.isEmpty() && heights[stack.peek()] > h) {
            int height = heights[stack.pop()];

```

```

        int width = stack.isEmpty() ? i : i - stack.peek();
        maxArea = Math.max(maxArea, height * width);
    }
    stack.push(i);
}
return maxArea;
}

```

Min Stack

```

class MinStack {
    Deque<int[]> stack = new ArrayDeque<>(); // {value, min}
    public void push(int val) {
        int min = stack.isEmpty() ? val : Math.min(val, stack.peek()[1]);
        stack.push(new int[]{val, min});
    }
    public void pop() { stack.pop(); }
    public int top() { return stack.peek()[0]; }
    public int getMin() { return stack.peek()[1]; }
}

```

Decode String (nested structure)

```

public String decodeString(String s) {
    Deque<Integer> counts = new ArrayDeque<>();
    Deque<StringBuilder> parts = new ArrayDeque<>();
    StringBuilder cur = new StringBuilder();
    int num = 0;
    for (char c : s.toCharArray()) {
        if (Character.isDigit(c)) {
            num = num * 10 + (c - '0');
        } else if (c == '[') {
            counts.push(num); parts.push(cur);
            num = 0; cur = new StringBuilder();
        } else if (c == ']') {
            StringBuilder decoded = cur;
            cur = parts.pop();
            int k = counts.pop();
            for (int i = 0; i < k; i++) cur.append(decoded);
        }
    }
    return cur.toString();
}

```

```
        } else {  
            cur.append(c);  
        }  
    }  
    return cur.toString();  
}
```

Top 20 Must-Do Stack Problems

1. LC 20 — Valid Parentheses
2. LC 155 — Min Stack
3. LC 150 — Evaluate Reverse Polish Notation
4. LC 739 — Daily Temperatures
5. LC 496 — Next Greater Element I
6. LC 503 — Next Greater Element II
7. GfG — Next Greater Element
8. GfG — Next Smaller Element
9. GfG — Stock Span Problem
10. LC 907 — Sum of Subarray Minimums
11. LC 316 — Remove Duplicate Letters
12. LC 402 — Remove K Digits
13. LC 84 — Largest Rectangle in Histogram
14. LC 85 — Maximal Rectangle
15. LC 42 — Trapping Rain Water
16. LC 394 — Decode String
17. LC 224 — Basic Calculator
18. LC 227 — Basic Calculator II
19. LC 1249 — Minimum Remove to Make Valid Parentheses
20. LC 32 — Longest Valid Parentheses

Top 30 Interview Problems

(The 20 above, plus:) 21. LC 71 — Simplify Path 22. LC 901 — Online Stock Span 23. GfG — The Celebrity Problem 24. GfG — Infix to Postfix

25. GfG — Sort a stack using recursion 26. GfG — Reverse a stack using recursion 27. LC 2487 — Remove Nodes From Linked List 28. LC 456 — 132 Pattern 29. LC 2454 — Next Greater Element IV 30. LC 636 — Exclusive Time of Functions

Beginner → Advanced Complete Sequence

Follow the Master Roadmap tables above, Level 0 → Level 8, Order 1 → 70. Within a level, problems are already sequenced so each builds on the one before it.

Final Summary

Total curated problems: 70 unique (across LeetCode + GfG; some are intentional cross-platform reps)

Platform	Count
LeetCode	48
GeeksforGeeks	22

Difficulty	Count (approx.)
Easy	14
Medium	41
Hard	15

Level / Pattern Group	# of Problems
Level 0 — Fundamentals & Basic Ops	5
Level 1 — Basic Easy (Stack + Arrays)	3

Level 2 — Stack + Strings	8
Level 3 — Parentheses / Expression	17
Level 4 — Monotonic Stack Foundations	7
Level 5 — Next Greater/Smaller Applied	10
Level 6 — Design / Recursion+Stack / HashMap / LinkedList	11
Level 7 — Histogram / Rain Water	6
Level 8 — Advanced/Hard	3 (+4 cross-ref)

Tag	Count (approx.)
 Must-Do	27
☆ Recommended	22
 Optional	12
 Hard Challenge	6

Study Plans

7-Day Plan (most important problems only): Day 1: 20, 225, 232, 155 · Day 2: 1047, 71, 394 · Day 3: 1249, 32, 150 · Day 4: 227, 224 · Day 5: 739, 496, 503, GfG Next Greater/Smaller Element · Day 6: GfG Stock Span, 907, 316, 402 · Day 7: 84, 85, 42

14-Day Plan (essential interview problems): Days 1–7: the 7-day plan above. Day 8: 682, 844, 946, 1209 · Day 9: 921, 1541, 1963, 856 · Day 10: GfG Infix to Postfix + conversions · Day 11: 901, 2104, 1130, 1673 · Day 12: GfG Celebrity Problem, GfG Sort/Reverse Stack Recursion · Day 13: 2487, 456, 636 · Day 14: full review — re-solve #20, #155, #739, #84, #42, #224 from scratch, timed.

30-Day Plan (strong preparation): Days 1–14: the 14-day plan above.

Days 15–20: 1544, 1190, 388, 385, 341, 1381, 716. Days 21–25: 962, 1856, GfG Next Greater Frequency Element, 173. Days 26–29: 1944, 2454, 772, 2281. Day 30: mixed timed review — 5 random Must-Do + 2 random Hard Challenge problems, no hints.

Final List — Most Important Problems to Re-Solve Before an Interview

If you only have time to re-drill a short list the night before an interview, this is it (one per core pattern, chosen so each forces recall of a distinct template):

1. **LC 20** — Valid Parentheses (basic stack mechanics)
 2. **LC 155** — Min Stack (auxiliary-state design)
 3. **LC 150** — Evaluate Reverse Polish Notation (postfix evaluation)
 4. **LC 739** — Daily Temperatures (monotonic decreasing stack)
 5. **GfG — Next Smaller Element** (monotonic increasing stack)
 6. **GfG — Stock Span Problem** (previous greater element applied)
 7. **LC 907** — Sum of Subarray Minimums (contribution technique)
 8. **LC 402** — Remove K Digits (greedy monotonic stack)
 9. **LC 84** — Largest Rectangle in Histogram (the hardest "core" template)
 10. **LC 42** — Trapping Rain Water (stack vs. two-pointer comparison)
 11. **LC 394** — Decode String (nested structure + recursion mindset)
 12. **LC 224** — Basic Calculator (expression evaluation with parens)
 13. **GfG — Reverse a stack using recursion** (recursion-as-implicit-stack)
-

Accuracy note: LeetCode numbers/titles were verified against LeetCode's own problem pages and cross-checked against multiple independent, current source listings. GfG problem titles were verified against GfG's practice-page listings and topic-tag pages at the time of writing. 🐉 Premium: LC 772 (Basic Calculator III). LC 716 (Max Stack) is marked with a "🐉?" in its row because its access tier has varied across sources at different times — verify current availability on LeetCode

before relying on it.